

Bluetooth Low Energy

BLE peripheral in Go, Flutter scan and connect, reconnection

Lecturer

Dinu-Ștefan Rusu

Course

MEC

Lab objectives

Run a peripheral

Advertise one service with one notifying characteristic from a Go program, on an nRF52840 or ESP32-C3 board or on a Linux or Windows laptop.

Write the central

Scan, connect, subscribe and show the value live in Flutter, and keep working after the peripheral disappears.

Time

2 hours

Present

To the lab instructor, at the end of this lab

Dependencies and file layout

1. Add the two packages:

```
flutter pub add flutter_blue_plus permission_handler
```

2. The peripheral lives in `peripheral/main.go`, in its own Go module.
3. The app screen lives in `lib/lab3/ble_screen.dart`, reached from your home screen.

Watch out

Neither the Android emulator nor the iOS simulator has a Bluetooth radio. Everything in this lab runs on a physical phone, over a cable.

Starting the peripheral

```
# Laptop: Linux or Windows
cd peripheral
go mod init lab3
go get tinygo.org/x/bluetooth
go run .

# Board: nRF52840, same source
tinygo flash -target=pca10056-s140v7 .
```

One source file, two ways to run it

The same program runs under `go run` on a laptop and on a board after `tinygo flash`. The Flutter side cannot tell the difference.

Watch out

The ESP32-C3 and ESP32-S3 work through espradio; a plain ESP32 needs an HCI co-processor. macOS is central-only, so a Mac cannot host the peripheral.

Peripheral, part 1: advertising

```
package main
import "time"
import "tinygo.org/x/bluetooth"
var adapter = bluetooth.DefaultAdapter
var value bluetooth.Characteristic
var svc, _ = bluetooth.ParseUUID("0000fff0-0000-1000-8000-00805f9b34fb")
var chr, _ = bluetooth.ParseUUID("0000fff1-0000-1000-8000-00805f9b34fb")

func main() {
    must(adapter.Enable()) // must panics on a non-nil error
    adv := adapter.DefaultAdvertisement()
    must(adv.Configure(bluetooth.AdvertisementOptions{
        LocalName: "MEC Lab 3", ServiceUUIDs: []bluetooth.UUID{svc}))
}
```

Peripheral, part 2: notifications

```
must(adv.Start())
must(adapter.AddService(&bluetooth.Service{UUID: svc,
    Characteristics: []bluetooth.CharacteristicConfig{{
        Handle: &value, UUID: chr,
        Flags: bluetooth.CharacteristicReadPermission |
            bluetooth.CharacteristicNotifyPermission}}}))
go func() {                                     // writing the handle notifies
    for i := byte(0); ; i++ {
        value.Write([]byte{i}); time.Sleep(time.Second)
    }
}()
select {}
}
```

Permissions and device scan

1. Declare the Android permissions in the manifest and the iOS usage string in Info.plist.
2. Request `BLUETOOTH_SCAN` and `BLUETOOTH_CONNECT` at runtime with `permission_handler`.
3. Wait for the adapter state to be on, then scan with `flutter_blue_plus`.
4. Show one row per device: name and RSSI, sorted by RSSI.

Done when

- ☐ A clean install shows the system dialog before any scan starts.
 - ☐ MEC Lab 3 appears in the list within five seconds of opening the screen.
 - ☐ The RSSI number changes when you walk away from the peripheral.
 - ☐ Denying the permission shows a message, not a crash.
-

Manifest and Info.plist entries

```
<!-- android/app/src/main/AndroidManifest.xml, inside <manifest> -->
<uses-permission android:name="android.permission.BLUETOOTH_SCAN"
                  android:usesPermissionFlags="neverForLocation"/>
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"
                  android:maxSdkVersion="30"/>

<!-- ios/Runner/Info.plist, inside the top-level <dict> -->
<key>NSBluetoothAlwaysUsageDescription</key>
<string>Shows the live reading from your lab peripheral.</string>
```

An undeclared Android permission is denied with no dialog. A missing iOS usage string terminates the app the moment it touches the Bluetooth API.

Request, wait for the adapter, scan

```
if (Platform.isAndroid) {  
  await [Permission.bluetoothScan,  
        Permission.bluetoothConnect].request();  
}  
await FlutterBluePlus.adapterState  
  .where((s) => s == BluetoothAdapterState.on).first;  
  
final sub = FlutterBluePlus.onScanResults  
  .listen((r) => setState(() => _found = r));  
FlutterBluePlus.cancelWhenScanComplete(sub);  
await FlutterBluePlus.startScan(  
  withServices: [Guid(kServiceUuid)], // drop once, to debug  
  timeout: const Duration(seconds: 15));
```

Connect, discover, and show the value

1. Tapping a row stops the scan and connects, with a ten-second timeout.
2. Discover the services and find the characteristic by its UUID.
3. Listen to the value stream, then call `setNotifyValue(true)`.
4. Show the decoded counter and the time of the last update.

Done when

- ☐ The number on screen increases by one every second.
 - ☐ No two updates in a row show the same number.
 - ☐ Leaving the screen stops notifications and disconnects.
 - ☐ The UUIDs appear once, as constants.
-

Listen first, then enable. Enabling notifications before the listener exists loses what arrives in between.

Find the characteristic and listen

```
await device.connect(timeout: const Duration(seconds: 10));
final services = await device.discoverServices();

final c = services
    .firstWhere((s) => s.uuid == Guid(kServiceUuid))
    .characteristics
    .firstWhere((x) => x.uuid == Guid(kCharUuid));

final sub = c.onValueReceived.listen(
    (v) => setState(() => _value = v.isEmpty ? null : v.first));
device.cancelWhenDisconnected(sub);

await c.setNotifyValue(true);    // writes the descriptor for you
```

Reconnection after a disconnect

1. Subscribe to `connectionState` before the first connect.
2. On a disconnect, clear the value and show *disconnected*.
3. Retry after a two-second delay, and keep retrying while the screen is open.
4. Rediscover the services after every reconnection.

Done when

- ☐ Killing the peripheral turns the screen to *disconnected* within five seconds.
 - ☐ No stale number is left on screen after the disconnect.
 - ☐ Restarting the peripheral resumes updates without restarting the app.
 - ☐ The retry survives three kill and restart cycles in a row.
-

Reconnect loop

```
device.connectionState.listen((s) async {
  if (s != BluetoothConnectionState.disconnected) return;
  setState(() { _value = null; _online = false; });
  await Future.delayed(const Duration(seconds: 2));
  if (mounted && _wanted) _tryConnect(); // _wanted: false in dispose
});

Future<void> _tryConnect() async {
  try {
    await device.connect(timeout: const Duration(seconds: 10));
    await _subscribe(); // rediscover: old objects are dead
  } catch (_) { /* the listener above schedules the next try */ }
}
```

Scanning and connecting

The device list stays empty

Permission denied, or location is off on Android 11 and earlier. Grant Nearby devices, switch location on.

connect, android-code: 133

The peripheral is not advertising, or another central still holds it. Restart it, wait two seconds, retry.

The app quits on the first scan on iOS

NSBluetoothAlwaysUsageDescription is missing. Add it, then rebuild: a hot restart does not reread Info.plist.

Every row in the list is unnamed

The advertisement carries no local name, or you read the wrong field. Use platformName.

No values received

Bad state: No element

A UUID in the app differs from the one in the Go file, or you reused services from the previous connection.

Connected, no value arrives

setNotifyValue(true) was never called, or the listener was added after it.

The counter moves, the phone is idle

A notification only reaches a subscribed central, and the characteristic needs the notify flag.

One value arrives, then nothing

The subscription died with the connection. Discover and subscribe again inside the reconnect path.

Peripheral errors

Bluetooth adapter not found

On Linux the service is down or the radio is blocked. Start bluetoothd, unblock with rfkill, run again.

It runs, nothing scans it

macOS is central-only in this package. Use Linux, Windows, or the board.

tinygo flash: unknown target

Use your board's target: pca10056-s140v7 (nRF52840 DK) or xiao-esp32c3.

Nothing works on an ESP32

Only the ESP32-C3 and ESP32-S3 advertise on their own, through espradio. A plain ESP32 needs an HCI co-processor: use the nRF52840 or a laptop instead.

Acceptance checks

Checked when you present

- ☐ `peripheral/main.go` builds and advertises.
 - ☐ The manifest and Info.plist entries are in place.
 - ☐ The scan list shows name and RSSI.
 - ☐ The live value updates once per second.
 - ☐ The peripheral is killed and restarted, and the app recovers.
-

How to present

On a physical phone, at the end of the lab. Be ready to name your two UUIDs and the byte encoding.

Before you leave

Run the whole path once from a fresh install of the app, with the phone unplugged from the laptop.

Presentation

Peripheral, scan, live value, reconnect: shown live